Lab 11: Government Data APIs

INFO 4617/5617 · Week 11 · Chapter 11, Government Data APIs

This week every class is live coding: one lab in three parts, one a day. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it.

We'll use two public APIs, and each one shows an idea that holds for APIs in general:

- **Part 1, Monday:** the **Census Bureau**, and what can I ask for?, starting from a table on its website;
- **Part 2, Wednesday:** more of the Census: comparing places, reading a margin of error, and the places the survey couldn't measure;
- **Part 3, Friday: FRED**, the Federal Reserve Bank of St. Louis's data service, and let the server compute.

Two more APIs are there to explore on your own, the EIA's and Congress.gov's; Step 13 says where to start.

Along the way, short ⏱️ microsprints ask you to change one thing, alone or in pairs, and share what you find.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Request two free keys and store them as environment variables **before you start Jupyter**. The chapter's "Setting Environment Variables" shows how; the simplest for this course stores them with the `webdata` environment, one line per key in a terminal (on Windows, the Anaconda Prompt), then a fresh start of Jupyter:

- `CENSUS_KEY`, from the Census Bureau, for Monday and Wednesday; click the activation link in its email before you use it;
- `FRED_KEY`, from FRED, for Friday.

Each part reads only the key it needs, and its setup step says what to do if it's missing.

On Wednesday and Friday, start at that day's setup step, or run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Rebuilt on 2026-10-28, after textbook #295 and the decision to keep the lab to two APIs: Wednesday is the Census alone (Steps 4–7), and Friday is FRED alone (Steps 8–13). The EIA left the lab, as Congress.gov did on 2026-10-26, and Step 13 points to both in the chapter. Each setup step's `getpass()` cell now asks only for a key that's missing, so **Run All** is safe, and Step 0 says how to use its box. Part 1, taught on 2026-10-26, keeps its code; its links, and Step 0's causes of a `KeyError`, follow the chapter's "When a Key Says `MISSING`". All three parts were run live on 2026-10-28 with `labs/tools/run_key.py`, and every cell ran, the `getpass()` cells included: with the keys set, they do nothing. The lab follows the textbook at commit `6ea83ac`. Part 1 and Step 7 use the 2020–2024 American Community Survey five-year estimates, so a rerun gives the same numbers; FRED's requests have fixed windows. The code is the chapter's without its `timeout`, `raise_for_status()`, and helper functions. Wednesday's part is about 33 minutes and Friday's about 35, before the 15-minute revision block.

Part 1 — Monday: the Census, and what can I ask for?

Every number on the Census Bureau's website also comes from its API. We start from one table on the website, find the same table in the API's catalog, read what it holds, and ask the API for the website's number. Finding out what an API offers, from the API itself, is called discovery.

Step 0 — Set up, and check your key (8 minutes)

An API key identifies you, counts your requests, and is yours alone. A key typed into a notebook travels with the notebook: into a GitHub repository, a class submission, a classmate's laptop. So treat it like a password: never paste it into a notebook cell or a code file. Read it instead from an environment variable, as the chapter's "Reading the Keys in Python" does, and check first that Python can see it, without printing it. Define the User-Agent too: these APIs ask for a key, but every request still says who sent it.

If it says `MISSING`, Python can't see the key, and `os.environ[...]` will stop with a `KeyError`. Jupyter sees only the environment variables that were set **before it started**, and only in **the place it was started from**. The chapter's "When a Key Says `MISSING`" lists the usual causes:

- Jupyter was already running when you ran `export`, or was started from somewhere else: Anaconda Navigator (which, on a Mac, doesn't read `~/.zshrc`), another terminal, or VS Code.
- `export` ran in a notebook cell, as `!export ...`: that sets the variable in a shell that closes at once, and leaves your key in the notebook.
- On Windows, which has no `export`, `setx` reaches only prompts opened after it.
- `conda env config vars set` ran, but the environment wasn't activated again.
- The name is different: `CENSUS_KEY`, in capitals, not `census_key` or `CENSUS_API_KEY`.

The dependable fix is the chapter's first way in "Setting Environment Variables": store the key with the `webdata` environment, `conda env config vars set CENSUS_KEY="your-census-key" -n webdata` in a terminal (on Windows, the Anaconda Prompt); then close that terminal, open a new one, run `conda activate webdata`, and start Jupyter from it. A Jupyter that was already running won't see the key until you quit it and start it again.

A second safe way keeps the keys in a file beside the notebook, `api_keys.json`, added to `.gitignore` **before your first commit**; the chapter's "Alternative: A JSON Configuration File" shows how to load it.

If you can't restart Jupyter now, let `getpass()` ask for the key. The step's second cell, given in full, does it, and only for a key that's missing: if the check said `set`, the cell does nothing, so **Run All** is safe.

1. Run the cell. A box opens under it (in VS Code, at the top of the window), with the words `CENSUS_KEY isn't set. Paste it here:`.
2. Click in the box, paste your key, and press **Enter**. Nothing shows as you paste, or only dots: that's on purpose, so the key never appears on screen or in the notebook.
3. Until you press Enter, the cell shows `[*]`, and no other cell will run. If the notebook seems stuck, look for the box.
4. Run the next cell, which reads the key. The key lasts until the kernel restarts; after a restart, run this cell again.

Paste the key only into the box, never between the cell's quotation marks: the words there are the question `getpass()` asks, and they're saved with the notebook.

```
# Import os, requests, pandas (as pd), and matplotlib.pyplot (as plt)
# Define HEADERS, whose "User-Agent" is "WebDataScience/1.0 (your-email@colorado.edu)", with your
↪  email
# Print whether "CENSUS_KEY" is in os.environ: "set" or "MISSING", never the key itself
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
print("CENSUS_KEY:", "set" if "CENSUS_KEY" in os.environ else "MISSING")
```

```
CENSUS_KEY: set
```

```
# Given: only if the check said MISSING, ask for the key with getpass (Step 0 says how). If it
 ↪  said set, this does nothing
from getpass import getpass
if not os.environ.get("CENSUS_KEY"):
    os.environ["CENSUS_KEY"] = getpass("CENSUS_KEY isn't set. Paste it here: ")
```

```
# Now it's set: read CENSUS_KEY into census_key with os.environ[...]
census_key = os.environ["CENSUS_KEY"]
```

💬 **Discuss**

- In week 10 we told Wikipedia who we are with a User-Agent. The Census Bureau also asks for a key: what can a key do that a User-Agent can't?
- Where could a key typed into this notebook end up: a commit, a screenshot, a shared notebook? The Census Bureau takes the key in the URL, so `response.url` contains it too. Which of those could you take back?
- Why does a terminal's `export` reach a Jupyter started from that terminal, but not one started from Anaconda Navigator?

Step 1 — From the website to the catalog: groups (10 minutes)

On data.census.gov, the Census Bureau's website for its tables, table **B19013** gives Boulder County's median household income: $103,994, ±$2,268, in the American Community Survey's (ACS) five-year estimates for 2020–2024 (the chapter's tour, figure 11.1). In the API, that table is a **group**, and the dataset lists all of its groups in a catalog of its own, `groups.json` (or, for people, groups.html). Load it into a DataFrame, tell pandas to show whole descriptions rather than cut them off, and look at a sample: each group has a `name`, the table's ID, and a `description`, its title. Then search the descriptions for the website's words, as the chapter's "From the Website's Table to the API's Group" does, with `str.contains()`.

```
# groups: a DataFrame of the JSON's "groups" from
 ↪  "https://api.census.gov/data/2024/acs/acs5/groups.json", its name and description
# Print how many groups there are, and show the first 10, sorted by name; set pandas'
 ↪  display.max_colwidth to None first
pd.set_option("display.max_colwidth", None)   # show whole descriptions, not "..."
groups_url = "https://api.census.gov/data/2024/acs/acs5/groups.json"
groups = pd.DataFrame(requests.get(groups_url, headers=HEADERS).json()["groups"])[["name",
 ↪  "description"]]
print(len(groups), "groups")
groups.sort_values("name").head(10)
```

```
1199 groups
```

	name	description
511	B01001	Sex by Age
135	B01001A	Sex by Age (White Alone)
122	B01001B	Sex by Age (Black or African American Alone)
124	B01001C	Sex by Age (American Indian and Alaska Native Alone)
126	B01001D	Sex by Age (Asian Alone)
129	B01001E	Sex by Age (Native Hawaiian and Other Pacific Islander Alone)
107	B01001F	Sex by Age (Some Other Race Alone)
111	B01001G	Sex by Age (Two or More Races)
116	B01001H	Sex by Age (White Alone, Not Hispanic or Latino)
119	B01001I	Sex by Age (Hispanic or Latino)

```
# Show the groups whose description contains "median household income" (case=False), sorted by
 ↪  name
groups[groups["description"].str.contains("median household income",
 ↪  case=False)].sort_values("name")
```

```
         name  \
1087   B19013
568   B19013A
560   B19013B
563   B19013C
574   B19013D
576   B19013E
570   B19013F
572   B19013G
532   B19013H
534   B19013I
1084   B19019
1193   B19049
1119   B22008
70     B25099
987    B25119
211    B29004
```

```
1087                                                                  Median Household Income in
 ↪ the Past 12 Months (in 2024 Inflation-Adjusted Dollars)
568                                      Median Household Income in the Past 12 Months (in 2024
 ↪ Inflation-Adjusted Dollars) (White Alone Householder)
560                     Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted
 ↪ Dollars) (Black or African American Alone Householder)
563            Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted Dollars)
 ↪ (American Indian and Alaska Native Alone Householder)
574                                      Median Household Income in the Past 12 Months (in 2024
 ↪ Inflation-Adjusted Dollars) (Asian Alone Householder)
576   Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted Dollars) (Native
 ↪ Hawaiian and Other Pacific Islander Alone Householder)
570                               Median Household Income in the Past 12 Months (in 2024
 ↪ Inflation-Adjusted Dollars) (Some Other Race Alone Householder)
572                                    Median Household Income in the Past 12 Months (in 2024
 ↪ Inflation-Adjusted Dollars) (Two or More Races Householder)
532                 Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted
 ↪ Dollars) (White Alone, Not Hispanic or Latino Householder)
534                                  Median Household Income in the Past 12 Months (in 2024
 ↪ Inflation-Adjusted Dollars) (Hispanic or Latino Householder)
1084                                                Median Household Income in the Past 12 Months
 ↪ (in 2024 Inflation-Adjusted Dollars) by Household Size
1193                                           Median Household Income in the Past 12 Months (in
 ↪ 2024 Inflation-Adjusted Dollars) by Age of Householder
```

```
1119           Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted
 ↪  Dollars) by Receipt of Food Stamps/SNAP in the Past 12 Months
70                                                Mortgage Status by Median Household Income in
 ↪  the Past 12 Months (in 2024 Inflation-Adjusted Dollars)
987                                                    Median Household Income in the Past 12
 ↪  Months (in 2024 Inflation-Adjusted Dollars) by Tenure
211                                Median Household Income for Households With a Citizen,
 ↪  Voting-Age Householder (in 2024 Inflation-Adjusted Dollars)
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Find a table for a question of your own.

Do: search the descriptions for another topic, such as ”median gross rent”, ”travel time to work”, or ”internet”. The key searches for ”median gross rent”. **Share:** your words, how many groups matched, and the one you'd use.

```python
# Microsprint: show the groups whose description contains your words (case=False), sorted by name
groups[groups["description"].str.contains("median gross rent", case=False)].sort_values("name")
```

```
       name  \
234  B25031
361  B25064
344  B25071
994  B25111
997  B25113

                                                                                description
234                                                                Median Gross Rent by Bedrooms
361                                                                  Median Gross Rent (Dollars)
344  Median Gross Rent as a Percentage of Household Income in the Past 12 Months (Dollars)
994                                                Median Gross Rent by Year Structure Built
997                                Median Gross Rent by Year Householder Moved Into Unit
```

💬 Discuss

- `B19013` came back with nine siblings, `B19013A` to `B19013I`. What do the letters add, and when would you want one of them instead?
- The website calls B19013 a table; the API calls it a group. What else did you have to translate from the website's words to the API's?
- Your search found several tables. How did you choose one?

Step 2 — A group's variables (8 minutes)

A group holds **variables**, the columns of the website's table, and each group has its own catalog: `groups/B19013.json` (or, for people, groups/B19013.html). A variable's name is the group, a line number, and a letter: `B19013_001E` is line 1's **E**stimate, and `B19013_001M` its **M**argin of error, the ”±” on the website. Names ending in `A` are annotations, which say when an estimate is missing or capped. Print each variable with its label, sorted by name.

```python
# variables: the JSON's "variables" from
 ↪  "https://api.census.gov/data/2024/acs/acs5/groups/B19013.json"
# Loop over sorted(variables.items()), and print each name and its "label"
variables = requests.get("https://api.census.gov/data/2024/acs/acs5/groups/B19013.json",
 ↪  headers=HEADERS).json()["variables"]
for name, about in sorted(variables.items()):
    print(name, "|", about["label"])
```

```
B19013_001E | Estimate!!Median household income in the past 12 months (in 2024 inflation-adjusted
 ↪  dollars)
B19013_001EA | Annotation of Estimate!!Median household income in the past 12 months (in 2024
 ↪  inflation-adjusted dollars)
B19013_001M | Margin of Error!!Median household income in the past 12 months (in 2024
 ↪  inflation-adjusted dollars)
B19013_001MA | Annotation of Margin of Error!!Median household income in the past 12 months (in
 ↪  2024 inflation-adjusted dollars)
GEO_ID | Geography
NAME | Geographic Area Name
```

⏱️ Microsprint (4 minutes, alone or in pairs)

A table with more lines.

Do: print the estimates (names ending in `E`) of group `B25034`, Year Structure Built, which we use on Wednesday. Each line is a decade. **Share:** how many lines it has, and what line `002` counts.

```python
# Microsprint: request groups/B25034.json; print each variable whose name ends in "E", with its
 ↪  label, sorted by name
housing_catalog = requests.get("https://api.census.gov/data/2024/acs/acs5/groups/B25034.json",
 ↪  headers=HEADERS).json()["variables"]
for name, about in sorted(housing_catalog.items()):
    if name.endswith("E"):
        print(name, "|", about["label"])
```

```
B25034_001E | Estimate!!Total:
B25034_002E | Estimate!!Total:!!Built 2020 or later
B25034_003E | Estimate!!Total:!!Built 2010 to 2019
B25034_004E | Estimate!!Total:!!Built 2000 to 2009
B25034_005E | Estimate!!Total:!!Built 1990 to 1999
B25034_006E | Estimate!!Total:!!Built 1980 to 1989
B25034_007E | Estimate!!Total:!!Built 1970 to 1979
B25034_008E | Estimate!!Total:!!Built 1960 to 1969
B25034_009E | Estimate!!Total:!!Built 1950 to 1959
B25034_010E | Estimate!!Total:!!Built 1940 to 1949
B25034_011E | Estimate!!Total:!!Built 1939 or earlier
NAME | Geographic Area Name
```

💬 Discuss

- The website shows the estimate and the margin side by side. In the API they are two variables. Why might an API split them?
- What do the `!!` in a label separate, and how would you get just the last part?
- Which of these variables would you ask for to rebuild the website's table exactly?

Step 3 — The website's number, from the API (10 minutes)

Now ask the API for what the website showed. A request names the variables with `get`, the place with `for`, and what that place sits inside with `in`, as the chapter's "The Same Numbers from the API" does. Places are named by FIPS codes, strings of digits: Colorado is state `08`, Boulder County is county `013` within it, and the city of Boulder is place `07850`. The answer is a list of lists whose first row is the header, and every value is a string, so make a DataFrame and convert the numbers with `pd.to_numeric`. It should match the website: $103,994, ±$2,268. Then ask for the city, first without `in`, and read what the API says. The chapter's "The Same Variable for a State, a County, and a City" adds Colorado itself, which needs only `for=state:08`.

```
# census_base = "https://api.census.gov/data/2024/acs/acs5"
# params: get="NAME,B19013_001E,B19013_001M", for="county:013", in="state:08", key=census_key
# Request it; county: a DataFrame from rows 1 onward, with row 0 as the columns; convert the two
 ↪  numbers; show it
census_base = "https://api.census.gov/data/2024/acs/acs5"
params = {"get": "NAME,B19013_001E,B19013_001M", "for": "county:013", "in": "state:08", "key":
 ↪  census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()
county = pd.DataFrame(data[1:], columns=data[0])
county[["B19013_001E", "B19013_001M"]] = county[["B19013_001E",
 ↪  "B19013_001M"]].apply(pd.to_numeric)
county
```

```
                     NAME  B19013_001E  B19013_001M state county
0  Boulder County, Colorado       103994         2268    08    013
```

```
# The city of Boulder is place 07850. Ask for it without "in": response = requests.get(...) with
 ↪  get, for="place:07850", and key
# Print response.status_code and response.text: what does the API say a place needs?
response = requests.get(census_base, params={"get": "NAME,B19013_001E", "for": "place:07850",
 ↪  "key": census_key}, headers=HEADERS)
print(response.status_code, response.text)
```

```
400 error: ambiguous geography "place:07850". You must either specify a wildcard or full qualify
 ↪  it with state
```

```
# Now with in="state:08": city, a DataFrame as above, with the two numbers converted; show it
params = {"get": "NAME,B19013_001E,B19013_001M", "for": "place:07850", "in": "state:08", "key":
 ↪  census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()
city = pd.DataFrame(data[1:], columns=data[0])
city[["B19013_001E", "B19013_001M"]] = city[["B19013_001E", "B19013_001M"]].apply(pd.to_numeric)
city
```

```
                    NAME  B19013_001E  B19013_001M state  place
0  Boulder city, Colorado        87493         6221    08  07850
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Another county.

Do: ask for the same two variables for another Colorado county; find its code in the FIPS code lists under "Counties". The key uses Larimer County, `069`. Then find the same county in table B19013 on data.census.gov. **Share:** the county, its median household income and margin, and whether the website agrees.

```
# Microsprint: set params["for"] to your county (the key: "county:069"), request, and show the
 ↪  DataFrame
params["for"] = "county:069"
data = requests.get(census_base, params=params, headers=HEADERS).json()
pd.DataFrame(data[1:], columns=data[0])
```

```
                     NAME B19013_001E B19013_001M state county
0  Larimer County, Colorado       93765        1936    08    069
```

💬 Discuss

- Did the API's number match the website's? If it hadn't, what would you check first: the variable, the place, or the year?

- Boulder County's median is $103,994 and the city's $87,493. How can a city's median be lower than its county's?
- Why must `08` stay a string? What would `int("08")` lose?

End of Part 1

On Wednesday, Part 2 compares places, checks a number against its margin, and finds the places with no estimate. It starts with its own setup step, which reads the key again.

Part 2 — Wednesday: more of the Census, and look before you trust

We finish what Monday started: comparing places, reading a margin of error, and the places the survey couldn't measure. All of it needs only the Census key.

Step 4 — Set up again (5 minutes)

Part 2 starts fresh, so a student who missed Monday can start here. Import what we need, define the User-Agent, check `CENSUS_KEY`, ask for it with `getpass()` only if it's missing (Step 0 says how to use the box), and read it.

```
# Import os, requests, pandas (as pd), and matplotlib.pyplot (as plt); define HEADERS as in Step 0, with your email
# Print whether "CENSUS_KEY" is set or MISSING
# census_base = "https://api.census.gov/data/2024/acs/acs5"
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
print("CENSUS_KEY:", "set" if "CENSUS_KEY" in os.environ else "MISSING")
census_base = "https://api.census.gov/data/2024/acs/acs5"
```

```
CENSUS_KEY: set
```

```
# Given: only if the check said MISSING, ask for the key with getpass (Step 0 says how). If it said set, this does nothing
from getpass import getpass
if not os.environ.get("CENSUS_KEY"):
    os.environ["CENSUS_KEY"] = getpass("CENSUS_KEY isn't set. Paste it here: ")
```

```
# Now it's set: read CENSUS_KEY into census_key with os.environ[...]
census_key = os.environ["CENSUS_KEY"]
```

💬 Discuss

- Did your key work today? If it didn't on Monday, which of Step 0's causes was it?
- **Run All Above** runs this step's `getpass()` cell too. Why doesn't it ask you for a key that's already set?

Step 5 — Compare two places (13 minutes)

Several places can go in one request: `place:07850,45970` is Boulder and Longmont, as in the chapter's "Comparing Places". Ask for all ten decades of group `B25034`, lines `002` (built 2020 or later) to `011` (1939 or earlier); a list comprehension with an f-string writes the ten codes. Then draw the chapter's figure

11.4: one pair of bars per decade, labeled with the words from the group's catalog, `groups/B25034.json` (Step 2's microsprint), and turned upside down first, so that time runs left to right, from the oldest decade to the newest. `DataFrame.plot.bar()` draws a pair for each row.

```
# housing_vars: "B25034_002E" to "B25034_011E", as [f"B25034_{line:03d}E" for line in range(2,
 ↪  12)]
# params: get="NAME," plus housing_vars joined with commas, for="place:07850,45970",
 ↪  in="state:08", key=census_key; request it
# df_compare: a DataFrame from rows 1 onward, with row 0 as the columns, housing_vars converted;
 ↪  show it
housing_vars = [f"B25034_{line:03d}E" for line in range(2, 12)]
params = {"get": "NAME," + ",".join(housing_vars), "for": "place:07850,45970", "in": "state:08",
 ↪  "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()
df_compare = pd.DataFrame(data[1:], columns=data[0])
df_compare[housing_vars] = df_compare[housing_vars].apply(pd.to_numeric)
df_compare
```

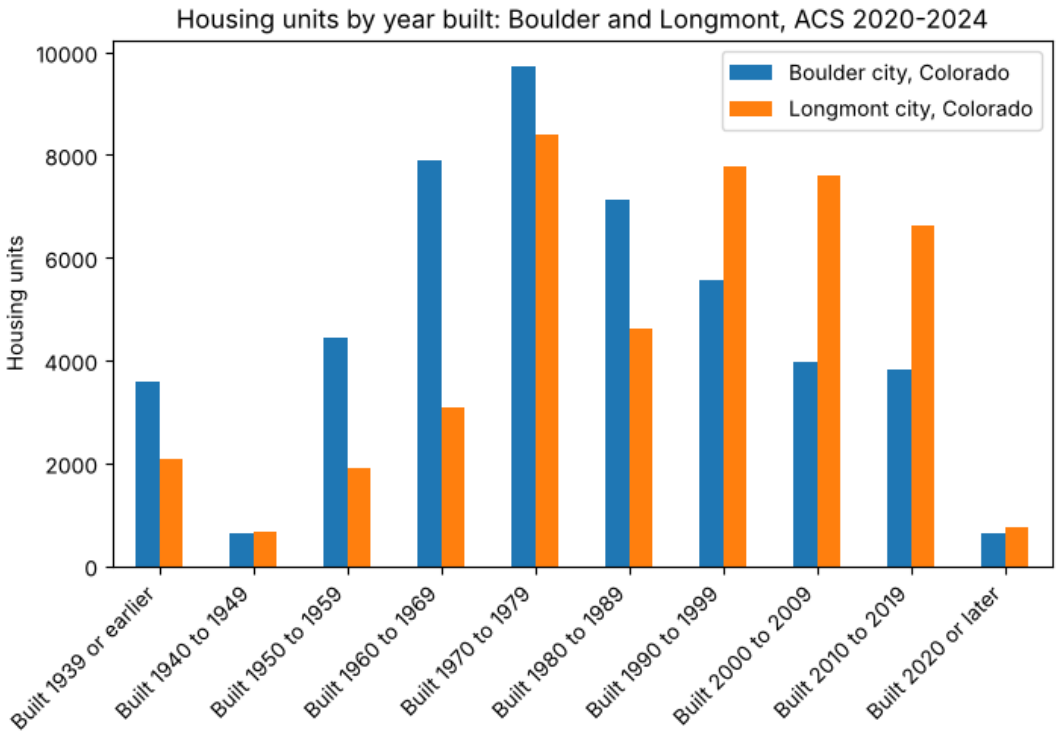
```
                     NAME  B25034_002E  B25034_003E  B25034_004E  \
0   Boulder city, Colorado          646         3836         3968
1  Longmont city, Colorado          761         6631         7597

   B25034_005E  B25034_006E  B25034_007E  B25034_008E  B25034_009E  \
0         5561         7141         9739         7904         4453
1         7781         4616         8402         3099         1899

   B25034_010E  B25034_011E state  place
0          641         3576    08  07850
1          671         2071    08  45970
```

```
# housing_catalog: the "variables" of groups/B25034.json; labels: each housing variable's label,
 ↪  the part after its last "!!"
# housing: df_compare indexed by NAME, its housing_vars renamed with labels, and transposed
# Plot housing.iloc[::-1] (oldest decade first) as bars; a title, a y label, no x label, a legend
 ↪  with no title
housing_catalog = requests.get("https://api.census.gov/data/2024/acs/acs5/groups/B25034.json",
 ↪  headers=HEADERS).json()["variables"]
labels = {var: housing_catalog[var]["label"].split("!!")[-1] for var in housing_vars}
housing = df_compare.set_index("NAME")[housing_vars].rename(columns=labels).T

ax = housing.iloc[::-1].plot.bar(figsize=(8, 4.5))
ax.set(title="Housing units by year built: Boulder and Longmont, ACS 2020-2024", ylabel="Housing
 ↪  units", xlabel="")
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha="right")
ax.legend(title="")
plt.show()
```



⏱️ Microsprint (6 minutes, alone or in pairs)

A short version of Additional Exercise 1. Cities differ in size, so raw counts don't compare fairly; shares do.

Do: ask for the total (`B25034_001E`) and the three decades since 2000 (`_002E` to `_004E`) for five Colorado places, and compute the share of each city's housing built in 2000 or later. The key uses Boulder, Longmont, Fort Collins (`27425`), Lakewood (`43000`), and Castle Rock (`12415`); look up others in the FIPS code lists. **Share:** the city with the largest share, the one with the smallest, and the two shares.

```
# Microsprint: set params["get"] to NAME and B25034_001E to B25034_004E, params["for"] to five
 ↪  places, and request
# df_five: indexed by NAME, the four counts converted with pd.to_numeric
# since_2000: 002E + 003E + 004E; share: since_2000 / 001E; show share, largest first
params["get"] = "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E"
params["for"] = "place:07850,45970,27425,43000,12415"
data = requests.get(census_base, params=params, headers=HEADERS).json()
df_five = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
df_five = df_five[["B25034_001E", "B25034_002E", "B25034_003E",
 ↪  "B25034_004E"]].apply(pd.to_numeric)
since_2000 = df_five["B25034_002E"] + df_five["B25034_003E"] + df_five["B25034_004E"]
share = since_2000 / df_five["B25034_001E"]
share.sort_values(ascending=False).round(3)
```

```
NAME
Castle Rock town, Colorado     0.712
Fort Collins city, Colorado    0.365
Longmont city, Colorado        0.344
Lakewood city, Colorado        0.182
Boulder city, Colorado         0.178
dtype: float64
```

💬 Discuss

- Which of your five cities built the most of its housing since 2000, and which the least? What might explain the difference?
- Boulder's biggest decade was the 1970s, and Longmont has more units than Boulder from every decade since the 1990s. The chapter names one likely reason, Boulder's 1976 cap on building permits, the Danish Plan. What would you need to know to say it was the reason?
- The chart counts the units standing in 2020–2024, by the year they were built. What does it leave out, and what does a five-year estimate mean for a count labeled "Built 2020 or later"?

Step 6 — Check a number against its margin (6 minutes)

A survey's estimate comes with a margin of error, the variable with the same name ending in `M`; the ACS gives it at 90% confidence. Two estimates whose ranges overlap can't be told apart. Ask for the median household income and its margin for Boulder and Longmont, and compute each city's range: the estimate minus and plus its margin. The chapter's "The Same Variable for a State, a County, and a City" compares the city's margin with the county's and the state's.

```
# params: get="NAME,B19013_001E,B19013_001M", for="place:07850,45970", in="state:08",
 ↪  key=census_key; request it
# incomes: indexed by NAME, the two columns converted; add low (estimate - margin) and high
 ↪  (estimate + margin); show it
params = {"get": "NAME,B19013_001E,B19013_001M", "for": "place:07850,45970", "in": "state:08",
 ↪  "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()
incomes = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
incomes = incomes[["B19013_001E", "B19013_001M"]].apply(pd.to_numeric)
incomes["low"] = incomes["B19013_001E"] - incomes["B19013_001M"]
incomes["high"] = incomes["B19013_001E"] + incomes["B19013_001M"]
incomes
```

```
                        B19013_001E  B19013_001M    low   high
NAME
Boulder city, Colorado         87493         6221  81272  93714
Longmont city, Colorado        90671         2975  87696  93646
```

💬 Discuss

- Do the two ranges overlap? What can you say, and not say, about which city's households earn more?
- Boulder's margin is twice Longmont's. What about the city could make its estimate less certain?
- Which checks like this one could a pipeline run on its own, every time it runs?

Step 7 — When there's no estimate (9 minutes)

A wildcard asks for every place of a kind: `for=place:*` with `in=state:08` returns all of Colorado's cities, towns, and census-designated places, as in the chapter's "Every Place in a State, and a Number That Isn't". Ask for each one's median household income, and `describe()` the column before you trust it: look at its minimum and its maximum.

Eighty-three of the 482 places have no estimate in the 2020–2024 survey (72 did in 2018–2022): too few households answered for the Census Bureau to compute one. The API doesn't leave the value out. It sends a number that can't be an income, `-666666666`, and `-222222222` for its margin of error; the Bureau's notes on estimate and annotation values list what each code means. Then look closely at four places: ask for the population (`B01003_001E`), the median income, and its margin for Boulder and three small towns, Bonanza (`07571`), Campo (`11645`), and Carbonate (`12030`), and set every negative value aside with `mask()`, which turns it into a missing value, `NaN`.

```
# params: get="NAME,B19013_001E", for="place:*", in="state:08", key=census_key; request it
# places: a DataFrame of the rows; places["income"]: B19013_001E converted with pd.to_numeric
```

```
# Print how many places came back, and describe() the income column
params = {"get": "NAME,B19013_001E", "for": "place:*", "in": "state:08", "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()
places = pd.DataFrame(data[1:], columns=data[0])
places["income"] = pd.to_numeric(places["B19013_001E"])
print(len(places), "places")
places["income"].describe().round()
```

```
482 places
```

```
count          482.0
mean    -114723934.0
std      251998296.0
min     -666666666.0
25%          43678.0
50%          72168.0
75%         106214.0
max         250001.0
Name: income, dtype: float64
```

```
# params: get="NAME,B01003_001E,B19013_001E,B19013_001M", for="place:07850,07571,11645,12030",
 ↪  in="state:08", key=census_key
# Request it; towns: indexed by NAME, the three columns converted with pd.to_numeric; show towns
params = {"get": "NAME,B01003_001E,B19013_001E,B19013_001M",
          "for": "place:07850,07571,11645,12030", "in": "state:08", "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()
towns = pd.DataFrame(data[1:], columns=data[0]).set_index("NAME")
towns = towns[["B01003_001E", "B19013_001E", "B19013_001M"]].apply(pd.to_numeric)
towns
```

```
                        B01003_001E  B19013_001E  B19013_001M
NAME
Bonanza town, Colorado           20   -666666666   -222222222
Boulder city, Colorado       106433        87493         6221
Campo town, Colorado             58        53125        40478
Carbonate town, Colorado          0   -666666666   -222222222
```

```
# Print the mean of the median-income column as it is; then mask every value below zero, and
 ↪  print the mean again
print(f"Mean with the codes in: {towns['B19013_001E'].mean():,.0f}")
towns = towns.mask(towns < 0)
print(f"Mean with them masked: {towns['B19013_001E'].mean():,.0f}")
towns
```

```
Mean with the codes in: -333,298,178
Mean with them masked: 70,309
```

```
                        B01003_001E  B19013_001E  B19013_001M
NAME
Bonanza town, Colorado           20          NaN          NaN
Boulder city, Colorado       106433      87493.0       6221.0
Campo town, Colorado             58      53125.0      40478.0
Carbonate town, Colorado          0          NaN          NaN
```

💬 Discuss

- `describe()` showed both ends of the scale: `-666666666`, and a largest median of 250,001. One is a code for no estimate, and one is a cap: "$250,000 or more". What would each do to a ranking of places by income, and would you have noticed?

- Why might an API send a code like this rather than leave the value empty? What does a missing value lose that the code keeps?
- Campo has an estimate this time, with a margin of error almost as large. Is that more useful than Bonanza's code, or less?
- Carbonate has no residents, yet it has a row. What does the Census count as a place, and what does that mean for a table of "every place"?

End of Part 2

On Friday, Part 3 turns to FRED, which computes changes, percent changes, and yearly averages for us before it sends them. It starts with its own setup step, which reads the FRED key, so you don't need today's cells to have run.

Part 3 — Friday: FRED, and let the server compute

FRED (Federal Reserve Economic Data), from the Federal Reserve Bank of St. Louis, holds 800,000 economic series, and it can transform one before it sends it: a change, a percent change, a lower frequency. Asking the server, instead of downloading everything and doing it in pandas, is today's idea.

Step 8 — Set up again (3 minutes)

Part 3 starts fresh. Import what we need, define the User-Agent, check `FRED_KEY`, ask for it with `getpass()` only if it's missing (Step 0 says how to use the box), and read it.

```
# Import os, requests, pandas (as pd), and matplotlib.pyplot (as plt); define HEADERS as in Step
 ↪  0, with your email
# Print whether "FRED_KEY" is set or MISSING
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
print("FRED_KEY:", "set" if "FRED_KEY" in os.environ else "MISSING")
```

```
FRED_KEY: set
```

```
# Given: only if the check said MISSING, ask for the key with getpass (Step 0 says how). If it
 ↪  said set, this does nothing
from getpass import getpass
if not os.environ.get("FRED_KEY"):
    os.environ["FRED_KEY"] = getpass("FRED_KEY isn't set. Paste it here: ")
```

```
# Now it's set: read FRED_KEY into fred_key with os.environ[...]
fred_key = os.environ["FRED_KEY"]
```

💬 Discuss

- The Census Bureau's key went in a URL parameter called `key`. FRED's is `api_key`. Why might every API name the same thing differently?

Step 9 — A FRED series, and what it is (6 minutes)

FRED (Federal Reserve Economic Data) names each of its 800,000 series with an ID, which you find on the series' web page (the chapter's tour, figure 11.2). Before downloading a series, ask the API's

`series` endpoint what it is. Then ask the `series/observations` endpoint for its values. FRED sends every value as a string and writes a missing one as `"."`, so convert them with `pd.to_numeric(errors="coerce")`, which turns `"."` into `NaN`. The series is Boulder County's median household income, as in the chapter's "Finding Series IDs".

```
# fred_base = "https://api.stlouisfed.org/fred"; params: series_id="MHICO08013A052NCEN",
 ↪  api_key=fred_key, file_type="json"
# Request f"{fred_base}/series" with params and HEADERS; info: the JSON's "seriess"[0]; print its
 ↪  title, frequency, and units
fred_base = "https://api.stlouisfed.org/fred"
params = {"series_id": "MHICO08013A052NCEN", "api_key": fred_key, "file_type": "json"}
info = requests.get(f"{fred_base}/series", params=params, headers=HEADERS).json()["seriess"][0]
print(info["title"], "|", info["frequency"], "|", info["units"])
```

```
Estimate of Median Household Income for Boulder County, CO | Annual | Dollars
```

```
# Add observation_end="2024-12-31" to params; request f"{fred_base}/series/observations"
# income: a DataFrame of the JSON's "observations", date and value only; convert both; show the
 ↪  last 3 rows
params["observation_end"] = "2024-12-31"
data = requests.get(f"{fred_base}/series/observations", params=params, headers=HEADERS).json()
income = pd.DataFrame(data["observations"])[["date", "value"]]
income["date"] = pd.to_datetime(income["date"])
income["value"] = pd.to_numeric(income["value"], errors="coerce")
income.tail(3)
```

```
         date     value
33 2022-01-01   96584.0
34 2023-01-01   97170.0
35 2024-01-01  103293.0
```

💬 Discuss

- The Census API answered with a list of lists. What shape is FRED's answer, and which is easier to turn into a DataFrame?
- The Census Bureau's estimate for Boulder County in 2020–2024 was $103,994; FRED's for 2024 is $103,293. The chapter says they come from different programs. Which would you cite, and how would you say which it is?
- Why fix `observation_end`, when the series only goes to 2024 anyway?

Step 10 — Let the server compute: `units` (8 minutes)

We'll ask FRED for observations several more times today, each time with one parameter changed. So gather Step 9's lines into one function, `fred_observations()`, the chapter's name: whatever changes from call to call is an argument, and `**params` passes along any other parameters. Then ask for the same series twice more, with `units` set to `"chg"`, the change in dollars from the year before, and to `"pch"`, the percent change, as the chapter's "Let the Server Compute: `units`" does. One series, three answers, and FRED computed two of them.

```
# Define fred_observations(series_id, **params): Step 9's observation lines, gathered
#   Add series_id, api_key, and file_type to params; request; build and convert the DataFrame;
 ↪  return it
def fred_observations(series_id, **params):
    params.update(series_id=series_id, api_key=fred_key, file_type="json")
    data = requests.get(f"{fred_base}/series/observations", params=params,
 ↪  headers=HEADERS).json()
    df = pd.DataFrame(data["observations"])[["date", "value"]]
    df["date"] = pd.to_datetime(df["date"])
```

```
    df["value"] = pd.to_numeric(df["value"], errors="coerce")
    return df
```

```
# change: fred_observations() with units="chg"; growth: with units="pch"; both with
 ↪  observation_end="2024-12-31"
# views: income's date's year and value, change's value, and growth's value as four columns; show
 ↪  the last 3 rows
change = fred_observations("MHICO08013A052NCEN", units="chg", observation_end="2024-12-31")
growth = fred_observations("MHICO08013A052NCEN", units="pch", observation_end="2024-12-31")
views = pd.DataFrame({"year": income["date"].dt.year, "dollars": income["value"],
                      "change": change["value"], "percent change": growth["value"]})
views.tail(3).round(1)
```

```
    year   dollars  change  percent change
33  2022   96584.0  6257.0             6.9
34  2023   97170.0   586.0             0.6
35  2024  103293.0  6123.0             6.3
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Check what the server computed against the dollars.

Do: find the years in which Boulder County's median household income fell: the rows of `views` whose `change` is below zero. **Share:** how many years, which ones, and the largest fall in dollars.

```
# Microsprint: falls, the rows of views whose "change" is below zero; show them
falls = views[views["change"] < 0]
falls.round(1)
```

```
    year  dollars  change  percent change
12  2001  57079.0 -1522.0            -2.6
13  2002  55810.0 -1269.0            -2.2
20  2009  63757.0 -3003.0            -4.5
21  2010  62215.0 -1542.0            -2.4
23  2012  66894.0 -1207.0            -1.8
31  2020  88289.0   -52.0            -0.1
```

💬 Discuss

- Did the falls cluster around any events you know? What would you check before saying they were caused by them?
- FRED computed `change` from the same dollars we have. When is it worth asking the server for something you could compute yourself?
- `fred_observations()` has no error handling: if FRED answered with a maintenance page, which line would fail, and what would its error say?

Step 11 — Let the server compute: `frequency` and `aggregation_method` (10 minutes)

A series' frequency is how often it is measured, and FRED can lower it before it sends the data, as the chapter's "Let the Server Compute: `frequency` and `aggregation_method`" shows. Take one series through three requests: `CPIAUCNS`, the Consumer Price Index (CPI) for all urban consumers in the United States, which the Bureau of Labor Statistics (BLS) publishes every month. Ask for it as published; then with `frequency="a"` and `aggregation_method="avg"`, which turn each year's twelve months into their mean; then with `units="pc1"` as well, the percent change of each year's mean from the year before's, which is the year's inflation. FRED applies the frequency first, then the units.

```
# years: observation_start="2015-01-01", observation_end="2025-12-31"; monthly: CPIAUCNS as
 ↪  published
# annual: frequency="a", aggregation_method="avg"; inflation: the same, plus units="pc1"
# table: annual's year, annual's value, and inflation's value; show it
years = {"observation_start": "2015-01-01", "observation_end": "2025-12-31"}
monthly = fred_observations("CPIAUCNS", **years)
annual = fred_observations("CPIAUCNS", frequency="a", aggregation_method="avg", **years)
inflation = fred_observations("CPIAUCNS", frequency="a", aggregation_method="avg", units="pc1",
 ↪  **years)
table = pd.DataFrame({"year": annual["date"].dt.year, "mean index": annual["value"],
                      "inflation": inflation["value"]})
table.round(1)
```

```
    year  mean index  inflation
0   2015       237.0        0.1
1   2016       240.0        1.3
2   2017       245.1        2.1
3   2018       251.1        2.4
4   2019       255.7        1.8
5   2020       258.8        1.2
6   2021       271.0        4.7
7   2022       292.7        8.0
8   2023       304.7        4.1
9   2024       313.7        2.9
10  2025       321.9        2.6
```

⏱️ Microsprint (4 minutes, alone or in pairs)

Check the server's math, as the chapter does, on the same series.

Do: from `monthly`, compute each year's mean with `groupby()` on the year, then each year's percent change with `pct_change()` times 100. **Share:** your numbers for 2024 and 2025 beside FRED's, and why your 2015 is missing.

```
# Microsprint: by_hand: monthly's values grouped by the date's year, averaged
# check: by_hand and its pct_change() * 100 as two columns; show the last 3 rows
by_hand = monthly.groupby(monthly["date"].dt.year)["value"].mean()
check = pd.DataFrame({"mean index": by_hand, "inflation": by_hand.pct_change() * 100})
check.tail(3).round(1)
```

```
      mean index  inflation
date
2023       304.7        4.1
2024       313.7        2.9
2025       321.9        2.6
```

💬 Discuss

- Did your numbers match FRED's? If they hadn't, which would you trust, and how would you find out?
- Why can FRED give 2015's inflation when our pandas version can't?
- `aggregation_method` could also be `sum` or `eop`, the end of the period. Which makes sense for a price index, and which for something counted, like jobs added?

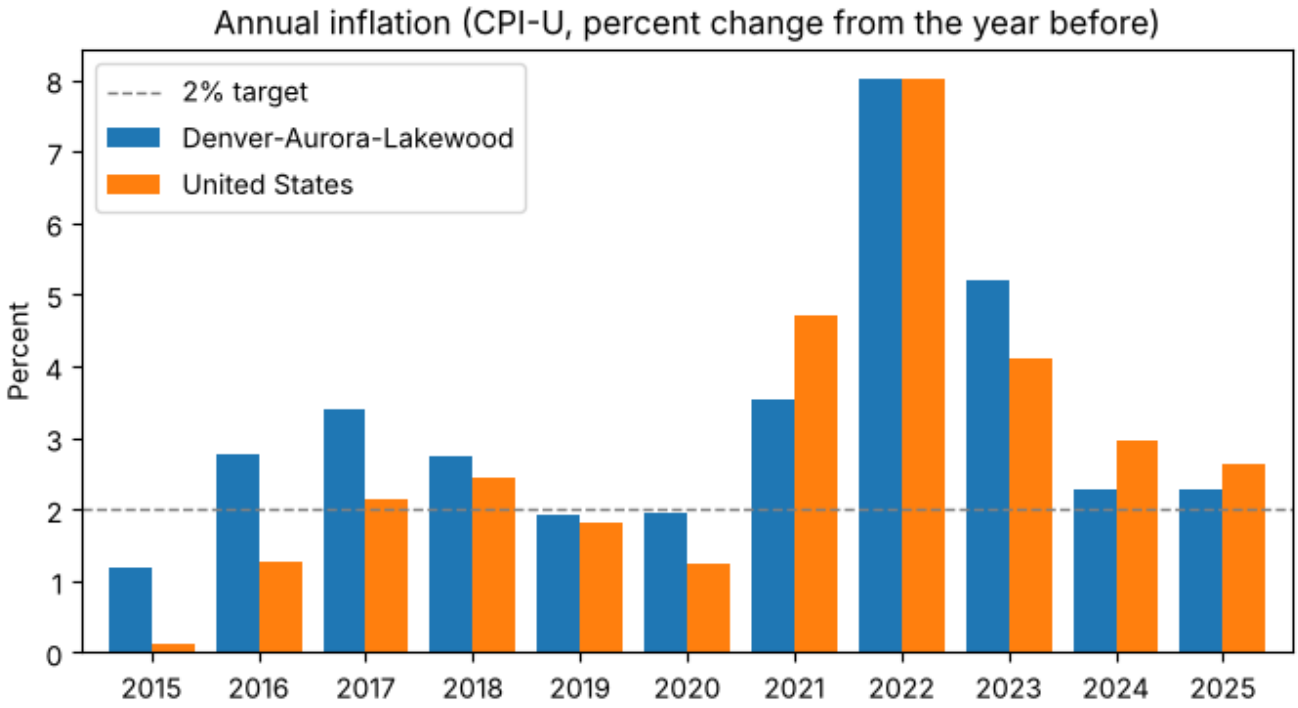
Step 12 — Denver against the nation (5 minutes)

With one series understood, comparing two is asking each the same question. The BLS (Bureau of Labor Statistics) also publishes a CPI (Consumer Price Index) for the Denver area, which FRED

carries as `CUUSA433SA0`. It is annual already, so it needs only `units="pc1"`. Merge it with Step 11's `inflation` on the date, and draw the chapter's paired bars: one pair a year, with a dashed line at 2 percent, the Federal Reserve's target, as in the chapter's figure 11.5. A DataFrame with one row per year and one column per place is what `DataFrame.plot.bar()` draws as pairs.

```
# denver: CUUSA433SA0 with units="pc1" and **years; cpi: denver merged with inflation on "date",
 ↪  suffixes ("_denver", "_national")
# pairs: cpi indexed by its date's year, the two value columns, renamed "Denver-Aurora-Lakewood"
 ↪  and "United States"
denver = fred_observations("CUUSA433SA0", units="pc1", **years)
cpi = denver.merge(inflation, on="date", suffixes=("_denver", "_national"))
pairs = cpi.set_index(cpi["date"].dt.year)[["value_denver", "value_national"]]
pairs.columns = ["Denver-Aurora-Lakewood", "United States"]
```

```
# Plot pairs as bars (rot=0); a dashed gray line at 2 labeled "2% target"; a title, a y label, no
 ↪  x label, and a legend
ax = pairs.plot.bar(figsize=(8, 4), rot=0, width=0.8)
ax.axhline(y=2, color="gray", linestyle="--", linewidth=1, label="2% target")
ax.set(title="Annual inflation (CPI-U, percent change from the year before)", ylabel="Percent",
 ↪  xlabel="")
ax.legend()
plt.show()
```



💬 Discuss

- In which years did Denver's prices rise faster than the nation's, and in which slower?
- Denver's series is annual and the nation's monthly. What would have gone wrong if we had merged the monthly series with the annual one, without `frequency="a"`?
- Housing is the largest part of the CPI. What would you ask FRED next to test whether rent explains the gap?

Step 13 — What did we learn? (3 minutes)

In a few sentences, as comments:

- Step 3 asked the API for the website's number. Did they match, and what would you check first if they hadn't?
- The week's two APIs each carried an idea: what can I ask for? and let the server compute. Where did each one save you work?
- In Step 7, how many of Colorado's places had no estimate, and what would their mean income have looked like with the codes left in?
- Which of this week's numbers would be different if you ran this notebook next week, and which wouldn't? Why?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, load your keys without printing them: Step 0's check, `"set" if ... in os.environ else "MISSING"`, its fixes for a `KeyError`, and `getpass()` when you can't restart.
- Step 2, an ACS variable for every Colorado county: Steps 1 and 2 to find the group and the variable, Step 3's `county:013` with `in="state:08"`, and Step 7's wildcard, which asked for every place.
- Step 3, build the DataFrame, convert, sort, and print the extremes: Step 3's `pd.to_numeric`, and Step 7's `describe()` and `mask()` for the codes that aren't numbers.
- Step 4, a bar chart of the top 20 counties: Step 5's bar chart.
- Step 5, a related FRED series for Colorado over time: Steps 9 and 10's `fred_observations()`; find the series on FRED's website, as the chapter's tour does.
- Step 6, interpret: the Discuss questions after Steps 3, 6, 7, and 11.

Further, on your own (not graded):

- **The EIA** (U.S. Energy Information Administration). The chapter's "EIA API v2" asks for Colorado's electricity prices and generation, and lets the server filter them before it sends them; "Combining Census and EIA Data" joins Wednesday's Census incomes with the EIA's electricity bills, state by state. Its key, `EIA_KEY`, comes from the EIA by email.
- **Congress.gov**, the Library of Congress's API for bills, members, and votes: entry 8 of the chapter's "More Government APIs for Colorado". A bill's record links to its actions, cosponsors, and text, and each link is a request you can send as it is. Its key comes from api.data.gov, and its documentation lists every endpoint.

```
# Your answers here, as comments
# 1. They matched: Boulder County's median household income came back as 103994 with a margin of
 ↪  2268, the website's
#    $103,994 and ±$2,268, and Larimer County's as 93765 and 1936. If they hadn't, I'd check the
 ↪  year first (the
#    dataset's address says 2024), then the variable (B19013_001E, not one of B19013A to
 ↪  B19013I), then the place codes.
# 2. "What can I ask for?" saved the searching: groups.json found B19013 from the website's own
 ↪  words, and
#    groups/B19013.json named its estimate and margin, B19013_001E and B19013_001M, before we
 ↪  asked for any data.
#    "Let the server compute" saved code: Step 10's units="pch" and Step 11's frequency="a" with
 ↪  units="pc1" replaced
#    a pct_change() and a groupby() we'd otherwise have written, and Step 11's microsprint showed
 ↪  they agree.
# 3. 83 of 482 places had no estimate. With their -666666666 codes left in, the mean of all 482
 ↪  was -114,723,934,
#    and the mean of our four towns -333,298,178; masked, the four towns' mean was 70,309.
# 4. The ACS 2020-2024 estimates won't change, and FRED's requests have fixed windows, so those
 ↪  numbers stay the same
#    unless FRED revises a series. The list of groups changes with each new ACS release.
```

Friday: from the lab to the textbook (15 minutes)

The last 15 minutes go to the textbook. Look back at what the lab turned up in chapter 11:

- **A cell that broke.** On Monday, `os.environ["CENSUS_KEY"]` stopped with a `KeyError` for some of us after `export`. Does the chapter's "When a Key Says `MISSING`" name your cause, and say clearly enough how to use `getpass()`? Where would you add a sentence?
- **A sentence that confused.** Which step needed the website beside the API to make sense? Does the chapter's tour do that for the Census table, and could it do more?
- **An output that has changed.** FRED revises some series. Compare Step 11's inflation and Step 12's Denver figures with the chapter's.
- **A section the lab skipped.** Can you follow the chapter's "EIA API v2" on your own, from the chapter alone? Where did you get stuck?
- **A microsprint that would make a good exercise.** Searching `groups.json` for your own topic (Step 1), or comparing two cities' margins (Step 6), could be an Additional Exercise.

The revision framework says how to propose a change, and what each kind needs to be accepted.